

Agent Loop 三方对比

v2 · ai 进化后

同一个「模型 ↔ 工具反复交互」内核——但 `ai` 这轮把 v1 标记的「可借鉴项」真的实现了：流式调用、流式并发工具、上下文压缩、一整套 IDE 级工具。

对象：`claudecode` (泄露源码) · `openclaw-2026.3.23` (内核引擎 Pi · earendil-works/pi) · `ai/` (本项目) · v2 生成日期：2026-06-24 (v1: 2026-06-23) · loop 分析据 Pi 源码修订：2026-06-29

- Claude Code — 软件工程 CLI, `claudecode/query.ts` (~1730 行单循环)
- OpenClaw — 多渠道私人助理, `src/agents/pi-embedded-runner/run.ts` (~1716 行编排壳)
- ai — 你的本地编码 agent, `ai/src/agent/engine.ts` (178 行) + `compact.ts` (103 行)

v1 → v2: `ai` 这轮补上了什么

v1 的结论是「ai 把循环还原成最朴素的样子，并按性价比挑 ①②④⑤ 来抄」。v2 已落地其中四项——而且没有牺牲可读性：循环主体仍是一个 `for` 生成器，只是从 85 行长到 178 行。

① 模型调用：非流式 → 流式 `llm.ts` · `streamCompletion`

新写的 `streamCompletion` 边收 SSE 边产出：`content` 片段实时吐 `delta`；并且能解析 `tool_calls` 的增量，在某个工具参数刚拼完整时立刻把它作为一次工具调用产出（依据「出现更大的 `tool index`」或「流结束」判定）。

③ 工具执行：顺序 `await` → 流式并发 `engine.ts` · `StreamingToolExecutor`

拿到一个工具调用就 `add()` 立即开跑（不 `await`），把 `Promise` 记下；本轮流结束后 `drain()` 按提交顺序回收结果。效果 = 模型还在吐后续 `token` / 后续工具调用时，先到的工具已经在并发执行——这正是 Claude Code 的 `StreamingToolExecutor` 思路。

② 上下文压缩：只会丢最旧 → 模型总结式 `autocompact` `agent/compact.ts`

每轮开头 `compactInPlace`：估算 `token` 超阈值（默认 40k）时，调一次模型把较旧那段历史总结成一条摘要，就地 `splice` 替换，只完整保留 `system` 与最近若干条；`cut` 对齐到 `user` 边界，绝不拆散 `assistant(tool_calls)` 与其 `tool` 结果。压缩失败静默回退，不影响主流程。

⑤ 内置工具：6 个 → IDE 级一整套 `tools.ts`

新增 `edit_file` (精确替换) · `glob` · `grep` · `web_fetch` · `run_agent` (子 agent)，与原有 `write/read/list/bash/email/stock` 合并，凑齐了 `Bash/Edit/Read/Write/Glob/Grep/Agent/Web` 这条 Claude Code 同款主线。

仍未补（且有意不补）：模型回退（单 `provider`，失败即抛）、逐工具权限闸（场景是单可信用户，系统提示直言「你有限权」）、多 `provider` 抽象层。这几项是为「无人值守 / 多模型 / 严格权限」付的税，ai 的场景用不上——继续留白以保住可读性。

一句话结论 (v2)

Claude Code 把循环本身做成产品。 一个 <code>while(true)</code> 生成器里塞进了全部上下文工程与容错：多级压缩、模型回	OpenClaw 自己不写循环，而是包住 Pi 的循环。 真正的「模型↔工具」迭代交给内嵌引擎 Pi (earendil-works/pi) ——一个 双层 <code>while</code> + <code>steering/follow-up</code> 的循环；	ai (v2) 朴素循环 + 三样关键升级，仍一眼读懂。 同一个 <code>for (step<25)</code> 生成器，现在每轮：压缩历史 → 流式调模型 → 工具边流
---	--	---

退、流式工具并发、对超长/超 token 的逐级恢复。终止有七八种 `reason`。

`run.ts` 是它外面的调度壳：会话队列、密钥轮换、模型回退、溢出补偿、多聊天渠道。

边并发跑 → 按序回灌。把 Claude Code 最值钱的三招（流式 / 流式并发 / 压缩）摘了过来，省下其余 1500 行的税。

同源，差距缩小。 三者内核仍完全一致——发消息 → 模型要么回话要么调工具 → 执行工具 → 结果塞回 → 再发。v1 里 ai 与 Claude Code 的差距在「流式 / 并发 / 压缩」三处最扎眼；v2 后这三处已对齐机制（只是层数更少、没有逐级恢复闸）。剩余差距主要是容错深度（模型回退、超长恢复、权限）。

核心循环（伪代码并排）

ai · v2 · 仍是一个 for 循环，但每轮多了压缩 / 流式 / 并发三件事

```
for (let step = 0; step < maxSteps; step++) { // 默认 25 步封顶
  await compactInPlace(history, deps) // ① 超阈值→模型总结式压缩（就地 splice，保留 system+近况）

  const exec = new StreamingToolExecutor(execTool) // 拿到工具就立即开跑的执行器
  let completion
  try {
    const stream = streamCompletion(history, { tools }) // ② 流式：content + tool_calls 都是增量
    for await (const part of stream) {
      if (part.type === 'text') yield { type:'delta', content: part.delta } // 文本边流边出
      else { yield { type:'tool', ... }; exec.add(part.call) } // ③ 参数一拼完整就并发开跑
    }
    completion = stream.return // { content, toolCalls, finishReason }
  } catch (e) { // ④ 被动恢复：上下文超长
    if (isContextOverflow(e) && !reactiveTried) { reactiveTried=true; await compactInPlace(history,deps,{force:true}); continue }
    throw e // 不可恢复（含用户 abort）→ 抛给上层反馈
  }
  const { content, toolCalls, finishReason } = completion
  history.push({ role:'assistant', content, tool_calls: toolCalls })

  if (exec.size) { // 有工具 → 回收并发结果，按序回灌，继续下一轮
    for await (const { call, result } of exec.drain())
      history.push({ role:'tool', tool_call_id: call.id, content: result })
    continue
  }
  if (finishReason === 'length' && outputRecovery<3) { // ⑤ 输出被截断 → 注入续写提示再来一轮
    outputRecovery++; history.push({ role:'user', content:'（系统）从断点继续，别重复别道歉' }); continue
  }
  return // ← 无工具且未被截断 = 最终答复，结束
}
yield { type:'text', content: `[已达最大步数 ${maxSteps}, 停止]` } // 兜底
ai/src/agent/engine.ts:90 · llm.ts:streamCompletion · agent/compact.ts
```

Claude Code · 同一个循环，外加每轮的上下文工程与逐级恢复

```
while (true) { // queryLoop — 单一长循环，靠 state 在迭代间传递
  messages = applyToolResultBudget(messages) // ① 工具结果按 tool_use_id 限额裁剪
  messages = snipCompact(messages) // ② 轻量删减
  messages = await microcompact(messages) // ③ 微压缩（编辑缓存，不重发）
  messages = applyCollapses(messages) // ④ context-collapse 投影视图
  ({ compactionResult } = await autocompact(messages)) // ⑤ 模型总结式压缩 ← ai 的压缩对标这一层
  if (atBlockingLimit()) return {reason:'blocking_limit'}

  for await (msg of callModel({messages, tools, fallbackModel})) {
    withhold(可恢复错误) // 暂扣 prompt-too-long / max-output-tokens
    streamingToolExecutor.addTool(toolBlock) // ⑥ 工具在「流式过程中」就开跑 ← ai 的 StreamingToolExecutor 对标这一招
  } // FallbackTriggeredError → 换模型重试整轮

  if (!needsFollowUp) { // 模型没要工具 → 准备收尾，但先过一串恢复闸
    if (withheld413) { collapseDrain() ?? reactiveCompact(); continue }
    if (maxOutputTok) { 8k-64k escalate ?? 注入恢复消息; continue }
    if (stopHook.blocks) { continue } // stop-hook 可挡住收尾
  }
}
```

```
    if (tokenBudget.continue) { continue } // +500k 自动续跑
    return {reason: 'completed'}
  }
  for await (r of runTools(blocks, canUseTool)) yield r // 权限闸 canUseTool 逐个放行 ← ai 仍无此闸
}
claudecode/query.ts:241 (queryLoop)
```

OpenClaw · Pi 引擎 · 双层循环：steering / follow-up 能把跑着的循环「续上」

```
while (true) { // 外层：本该收尾，被 follow-up 重新唤醒
  let hasMoreToolCalls = true
  while (hasMoreToolCalls || pending.length) { // 内层：一个个工具回合
    pending = await config.getSteeringMessages() ?? [] // ① steering：用户中途插话即时塞进上下文
    for (const m of pending) ctx.messages.push(m)
    const msg = await streamAssistantResponse(ctx) // ② 流式：text/thinking/tool_calls 边出 (多 provider 经 pi-ai 抽象)
    if (msg.stopReason === 'error' || msg.stopReason === 'aborted') break
    const calls = msg.toolCalls; hasMoreToolCalls = calls.length > 0
    const results = sequential
      ? await runSequential(calls)
      : await Promise.all(calls.map(run)) // ③ 流「结束后」才并行—不与流式重叠
    // beforeToolCall / afterToolCall 钩子在此校验·改写·重试 (openclaw 的策略闸挂这里)
    if (results.every(r => r.terminate)) break // 工具自带终止信号
    if (config.shouldStopAfterTurn?.()) break
  }
  const followUp = await config.getFollowUpMessages() ?? [] // ④ 要停了又来活 → 续跑
  if (followUp.length) { pending = followUp; continue }
  break
}
// 压缩在外层按需触发：contextTokens > contextWindow - reserveTokens(16384) → 模型总结旧史，留 system + 近 ~20k token + 完整 turn
earendil-works/pi · packages/agent/src/agent-loop.ts · coding-agent/src/core/compaction/compaction.ts
```

Pi 与另两家的两处真差异。① 工具并发的时机：Pi 是「流结束后」`Promise.all` 并行，而 Claude Code 与 ai v2 是「边流边跑」(`StreamingToolExecutor`，模型还在吐 token 工具已开跑)——同为并发，Pi 少了流式重叠那一层。② 循环可被外部「续上」：`getSteeringMessages` 让聊天渠道在循环跑着时插话、`getFollowUpMessages` 让它在本该收尾时再接新任务——这正是 OpenClaw 多渠道实时对话的底座；CC 与 ai 的循环一旦进入就闷头跑到收尾。

v2 后的对位关系。ai 的 `compactInPlace` ≈ CC 的 `autocompact` (仅这一层，没有 `budget/snip/micro/collapse` 那四层前置裁剪)；ai 的 `StreamingToolExecutor` ≈ CC 的 `streamingToolExecutor.addTool` (机制相同，但 ai 无 `canUseTool` 权限闸、无 `withhold` 恢复)。内核招式对齐，容错深度仍是 CC 厚得多。

逐维度对照 (v2)

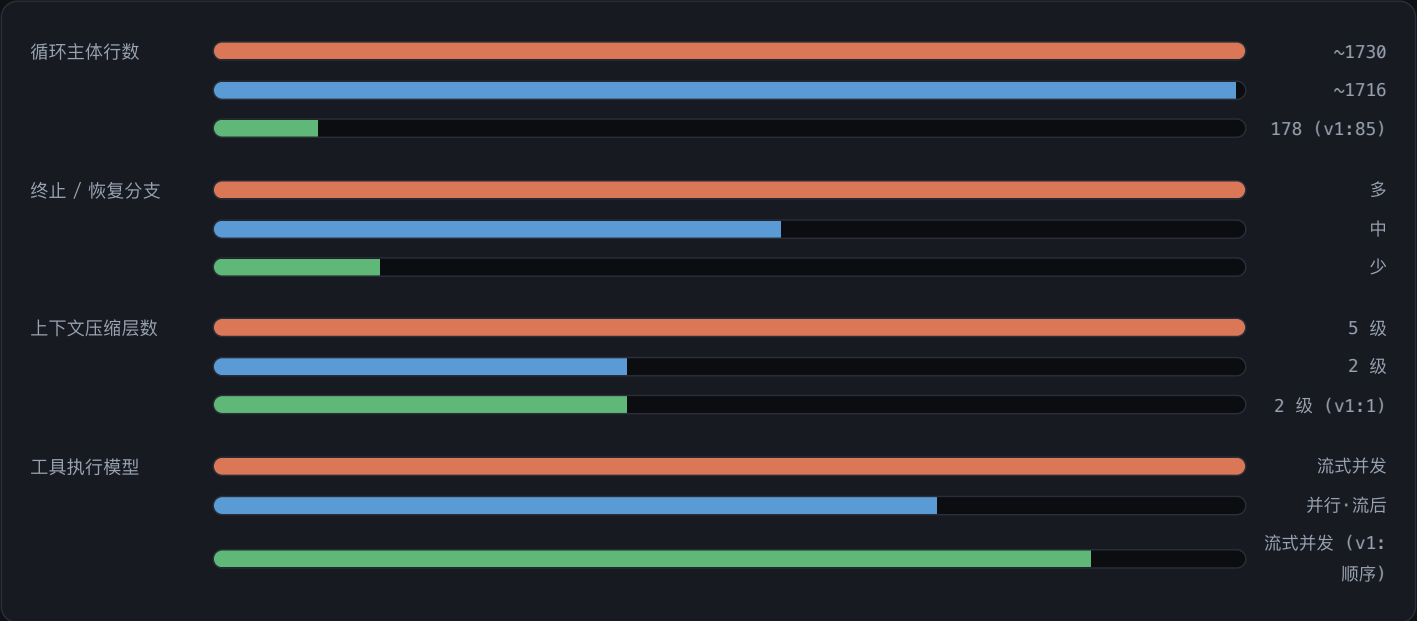
绿色 = 本版 ai 相比 v1 的变化。

维度	Claude Code	OpenClaw	ai (v2)
循环载体	<code>while(true)</code> async generator，单文件 1730 行	<code>run.ts</code> 外层 <code>while(true)</code> 调度壳；真正 agent 循环是 Pi 的双层 <code>while</code> (外层 <code>follow-up</code> 重入 · 内层工具回合)，见 <code>agent-loop.ts</code>	<code>for (step<25)</code> async generator 178 行 (v1: 85)
模型调用	流式 (SSE)， <code>callModel</code> 边流边产出	流式：Pi 的 <code>streamAssistantResponse</code> 产出 <code>text/thinking/tool_calls</code> ；多 provider 经 <code>pi-ai</code> 抽象	流式 <code>streamCompletion</code> : <code>content + tool_calls</code> 增量边解折边产出 (v1: 非流式 <code>chatComplete</code>)
协议	Anthropic Messages API (原生 <code>content blocks</code>)	多 provider (Anthropic / OpenAI / ..., 经 <code>pi-ai</code> 抽象)	任意 OpenAI 兼容 <code>/chat/completions</code> (默认 DeepSeek)

维度	Claude Code	OpenClaw	ai (v2)
终止条件	无 tool_use + 过七八道恢复闸 → <code>reason: completed</code> , 另有 <code>blocking_limit / aborted / model_error / prompt_too_long...</code>	Pi: <code>stopReason=error/aborted</code> · <code>shouldStopAfterTurn()</code> · 无工具且无 <code>pending</code> · 工具结果 <code>terminate=true</code> → 退内层; 外层 <code>getFollowUpMessages</code> 可再唤醒。openclaw 外壳另在溢出/认证/模型错误时重试	本轮无工具调用 (<code>exec.size===0</code>) → <code>return</code> ; step 达 25 → 兜底文字 ; 新增两道恢复闸: <code>finish_reason==='length'</code> 截断 → 注入续写重跑 (≤3 次)、API 报上下文超长 → 强制压缩重试 (一次性 <code>guard</code>); 用户 <code>abort</code> → 抛出由上层反馈
工具执行	流式并发: <code>StreamingToolExecutor</code> 在模型仍在输出时就开跑; <code>canUseTool</code> 权限闸	顺序 / 并行可配: <code>executeToolCalls</code> 默认 <code>Promise.all</code> 并行, 但都在流结束后才跑 (非流式并发); <code>beforeToolCall/afterToolCall</code> 钩子供 openclaw 挂策略闸 + 注入工具集	流式并发: <code>StreamingToolExecutor</code> , 工具参数拼完即 <code>add()</code> 开跑, <code>drain()</code> 按序回收 (v1: 顺序 <code>await</code>); 无权限闸
上下文压缩	五级: <code>budget 裁剪 / snip / microcompact / context-collapse / 模型总结 autocompact</code>	Pi auto-compaction: <code>contextTokens ></code> <code>contextWindow -</code> <code>reserveTokens(16384)</code> 时模型总结旧史, 保留 <code>system</code> + 最近 ~20k token + 完整 <code>turn</code> 边界 (文件操作单独追踪、旧摘要迭代更新); + openclaw <code>compactEmbeddedPiSession</code> 重试补偿	两级: 超 token 阈值 → 模型总结 <code>compactInPlace</code> (就地 <code>splice</code>); + <code>SessionStore.trim()</code> 超 60 条丢最旧 (v1: 仅 <code>trim</code>)
超长 / 超 token 恢复	逐级: <code>collapse-drain</code> → <code>reactive compact</code> ; <code>max-output 8k→64k 升档</code> →注入恢复消息	<code>contextOverflow</code> → 压缩后整轮重试 (带 <code>diag</code> 计数防死循环)	主动防膨胀 (每轮开头按阈值压缩) + 被动恢复: 输出截断 <code>length</code> → 续写重跑; API 上下文超长 → 强制压缩重跑一次 (<code>reactiveCompactAttempted</code> 防死循环) + 25 步封顶兜底 (v1: 仅主动压缩, 无报错后恢复)
用户中断	用户打断 → <code>aborted_streaming</code> / <code>aborted_tools</code> , 注入中断消息后退出	外层 <code>abortSignal</code> 贯穿; 渠道侧可取消	终端按 <code>Esc</code> 中断; QQ/微信任务进行中发「停 / 暂停 / 等一下 / stop」等 → <code>abort</code> 当前会话并回「已停止」(按会话持有 <code>AbortController</code>)
模型回退	有: <code>FallbackTriggeredError</code> 换 <code>fallbackModel</code> 重试整轮 + 清理孤儿消息	有: 模型回退链 + 认证 <code>profile</code> 冷却轮换	无 (仍是单 <code>provider</code> , 失败即抛)
多会话 / 并发	子 agent (<code>AgentTool</code>) 有独立 <code>queryTracking</code> 链; 主循环单会话	会话级 + 全局级命令队列 (<code>lane</code>) 串行化; 多聊天渠道并存	<code>SessionStore</code> Map: 私聊按用户、群按群各存一份历史 ; 新增 <code>run_agent</code> 子 agent (独立历史、深度上限 2)
扩展 / 钩子	<code>post-sampling / stop / stop-failure hooks</code> 、 <code>MCP</code> 、技能预取、记忆预取	插件系统、 <code>MCP</code> 、 <code>before/after compaction & tool-call</code> 钩子	channel 注入的 <code>extraTools</code> (如 QQ 的 <code>send_image</code>), 无钩子
权限	<code>canUseTool</code> 逐工具放行 + <code>permission mode</code> (含 <code>plan</code> 模式)	沙箱 / 工作区挂载路径限制 + 渠道白名单	无门禁——系统提示直言「你有权限操作设备, 别推辞」
内置工具	<code>Bash/Edit/Read/Write/Glob/Grep/Agent/Web...</code> 一整套 IDE 级	编码工具 + 渠道消息工具 (<code>sendMessage</code> 等) + 记忆 / 搜索	<code>Bash/Edit/Read/Write/Glob/Grep/Agent/Web 全套 + email / stock</code> (v1: 仅 <code>write/read/list/bash/email/stock</code> 六分)

复杂度量表 (v2 · 数量级直觉)

ai 的「循环主体行」从 85 涨到 178 (engine 178 + compact 103 ≈ 281)，仍是 CC/OC 的零头；但压缩与并发的「机制层数」明显追上来了。



行数为对应文件粗略规模；OpenClaw 的 1716 行是「编排壳」，真正 agent 循环在外部 Pi 引擎里，故其内核复杂度被低估。

同一请求下三者怎么跑 (v2)

请求：「读一下 config.ts，然后把里面的端口改成 8080」（需要：读文件 → 改文件，两轮工具）

1 compactInPlace
历史没超阈值 → 跳过

2 streamCompletion (流式)
文本边出 delta；tool_call: read_file
参数拼完 → 立即 exec.add 开跑

3 drain 回收 + 回灌
read_file 结果按序塞回 tool 消息

4 streamCompletion
这轮改用 edit_file 精确替换端口 → add → drain

5 streamCompletion
无 toolCalls (exec.size===0) → return

1 压缩 5 件套 + 预取
budget/snip/micro/collapse/autocompact

2 流式 callModel
Read 工具边流边执行

3 canUseTool 放行
Edit 命中权限 → 经确认后写入

4 收尾闸
stop-hook / token-budget 检查

5 reason: completed
每轮都重跑全部压缩与恢复检查

1 入队 (会话+全局 lane)
保证同会话不并发

2 runEmbeddedPiSession
Pi 双层循环：流式出 → 工具流后
Promise.all → shouldStopAfterTurn，跑完读→改两轮

3 attempt 返回
检查溢出 / 认证 / 模型错误

4 无异常 → break
否则压缩/换 key/换模型后重试

5 BlockChunker 整流
只把最终回复推给聊天渠道

● ai v2: 流式 + 工具边流边跑，改文件用 edit_file 而非整篇重写

● Claude Code: 同样两轮工具，外裹上下文工程

● OpenClaw: 循环细节被 Pi 封装，外层管调度

关键差异点的演化。v1 里 ai 的流程是「读→改」两轮纯顺序、无压缩、整篇 write_file 覆盖；v2 后变成「流式产出 + 工具边流边并发 + 改文件用 edit_file 精确替换 + 每轮先按阈值压缩」。流程图的步数没变（仍 5 步），但每一步内部的工程含量，已经从「裸调用」升到「带流式与压缩的小型 agent loop」。

file:///Users/lteu/progetto/agent/%E5%AF%B9%E6%AF%94-agent-loop-%E4%B8%89%E6%96%B9-v2.html

5/8

大文件 / 大文本怎么进上下文 (v2)

「读一个几 MB 的文件」「抓一个超长网页」是把上下文撑爆的最常见来源。三者都得在结果塞回 history 之前先做截断/分页，否则一次 read 就能顶满窗口。三家的取舍差得很明显：ai 一刀切头部、Claude Code 双闸 + 行级分页、OpenClaw 自适应分页 + 把超大外部内容挡在上下文之外先做摘要。

Claude Code

双闸 + 行级分页，宁可报错也不灌满。

读前 stat 卡总字节 (256KB) → 超了直接抛 FileTooLargeError；读后再卡输出 token (25000) → 超了抛错并提示「用 offset/limit」。默认只读前 2000 行、cat -n 带行号；<10MB 走内存 split，更大或 FIFO/设备走流式读。

OpenClaw

自适应分页 + 大外部内容先摘要。

包住 Pi 的 read：未指定 limit 时自动翻页 (最多 8 页)，跟着每页 truncation.outputLines 续读、拼接，攒到 maxBytes 为止；单次预算随模型上下文窗口自适应 (50KB→512KB)。超长链接/媒体则走 link/media-understanding，先用外部 CLI 抽取摘要再进上下文。

ai (v2)

一刀切头部 20000 字符。

最朴素：read_file 整文件读进内存，>20000 字符就 slice(0,20000)+'...' (已截断)，只给头部、无分页、无行号。grep 限 200 行命中、每行截 300 字；web_fetch 默认 20000 字。叠加的大结果靠下游 compactInPlace 兜底。

维度	Claude Code	OpenClaw	ai (v2)
单次读取上限	总字节 256KB (MAX_OUTPUT_SIZE)，读前 stat 闸) + 输出 25000 token (读后闸)	每次 50KB (DEFAULT_READ_PAGE_MAX_BYTES)，随上下文窗口放大到 ≤512KB	20000 字符 (写死在 read_file 里)
超限策略	抛错 (FileTooLargeError / token 超限)，让模型改用 offset+limit——刻意选「抛错」而非截断，省 token	自动翻页拼接到预算上限，再补一句 [Read output capped at NKB... Use offset=N to continue.]	头部截断，尾部贴「... (已截断)」，模型无从续读
分页 / 续读	原生 offset / limit 行级参数；默认前 2000 行	对模型隐藏：包装层自动翻页 (≤8 页)，也可显式传 offset/limit	无——只能整文件或换 run_bash 自己 sed -n
行号 / 格式	cat -n 行号；超长文件附「shorter than offset」「totalLines」提示	沿用 Pi 的行号输出；拼接时剥掉每页的续读提示行	原文照搬，无行号
超大文件路径	<10MB: readFile +内存 split；≥10MB / FIFO / 设备: createReadStream 流式逐行	交给 Pi 引擎；包装层只管字节预算与翻页	一律 readFileSync 全量进内存 (大文件有 OOM 风险)
重复读去重	有：内容未变 → 返回 FILE_UNCHANGED_STUB，不重发整文件	由 Pi 会话管理	无：每次都重读重灌
大外部内容	WebFetch 自带抓取+截断	link/media-understanding：外部 CLI 抽取摘要 (正文默认 500 字、CLI 缓冲 5MB) 后才进上下文	web_fetch 去标签转纯文本，截到 max_chars (默认 20000)
搜索结果 (grep)	GrepTool 限行 + ripgrep 后端	Pi 的 grep/glob	限 200 行命中、每行截 300 字、含 \0 的文件判为二进制跳过

ai · read_file —— 整文件进内存，头部一刀切

```
case 'read_file': {
  const path = resolve(String(args.path))
  const text = readFileSync(path, 'utf8') // ← 全量读入，无 offset/limit、无流式
  return text.length > 20000
    ? text.slice(0, 20000) + '\n... (已截断)' // ← 只给头部，模型无法续读后半段
    : text
}
```



```
}
ai/src/tools.ts:286
```

Claude Code · 读前卡字节、读后卡 token，行级分页

```
// 读前: stat 卡总文件大小 (仅当未显式给 limit 时)
if (!truncateOnByteLimit && maxBytes !== undefined && stats.size > maxBytes)
  throw new FileToolLargeError(stats.size, maxBytes) // MAX_OUTPUT_SIZE = 256KB

// 选路: 小文件内存 split, 大文件 / FIFO / 设备走流式
if (stats.isFile() && stats.size < 10*1024*1024) readFileInRangeFast(...)
else readFileInRangeStreaming(...) // createReadStream 逐行

// 读后: 实际输出 token 超限 → 抛错 (不截断, 省 token), 提示改用 offset/limit
await validateContentTokens(content, ext, maxTokens) // DEFAULT 25000
// 默认前 MAX_LINES_TO_READ=2000 行, cat -n 行号; 内容未变则回 FILE_UNCHANGED_STUB
claudecode/utils/readFileInRange.ts · tools/FileReadTool/limits.ts · prompt.ts
```

OpenClaw · 包住 Pi 的 read，自适应分页拼接

```
// 单次字节预算随模型上下文窗口自适应: 50KB → 512KB
maxBytes = clamp(ctxTokens * 4 * 0.2, 50*1024, 512*1024)

for (let page = 0; page < 8; page++) { // MAX_ADAPTIVE_READ_PAGES
  const r = await base.execute({ ...args, offset: nextOffset }) // 调 Pi 的 read
  const t = extractReadTruncationDetails(r)
  if (aggregatedBytes + nextBytes > maxBytes) { capped = true; break }
  aggregatedText += stripReadContinuationNotice(pageText) // 拼接, 剥掉每页续读提示
  if (!t?.truncated) break
  nextOffset += t.outputLines // 跟着上一页行数续读
}
if (capped) finalText += '\n\n[Read output capped at ${maxKB}KB.. Use offset=${n} to continue.]`
openclaw/src/agents/pi-tools.read.ts:208 (executeReadWithAdaptivePaging)
```

给 ai 的最小补法。不必抄 CC 的五层闸或 OC 的自适应分页，单是给 read_file 加一对 offset / limit (行号) 参数 + 默认只读前 N 行，就能让模型对大文件「先扫一段、要哪段读哪段」，而不是吃一个头部 20000 字的死截断后无路可走—— 几行改动，契合 ai 「几十行就对齐一招」的一贯取舍。再进一步可仿 CC 的 stat 预检，避免把几 MB 文件整个 readFileSync 进内存。

v2 之后，ai 还剩哪些可选项

v1 清单里的 ①②③⑤ 已落地 (流式 / 压缩 / 并发 / 工具集)。

本轮又补了 Claude Code 的两道恢复闸: 输出截断续写 (对标 max_output_tokens_recovery)、上下文超长强制压缩重试 (对标 reactive_compact_retry)，以及用户主动中断 (Esc / IM 发「停」)。

剩下按性价比:

改进点	现状	可借鉴谁 · 怎么做
④ 模型回退	单一 provider，请求失败直接抛错 (流式与非流式都是)	OC 的回退链: 主模型 5xx/限流时自动切备用 baseUrl/model，对多 provider 配置天然契合。流式下需处理「已吐一半再切」的清理。
⑥ 会话串行	QQ/微信已用 busy 集挡住同会话并发; 但仅「拒绝」而非「排队」	OC 的 lane 队列: 给每个 session 一把串行锁排队执行，而非直接回「上一条还在处理中」。
+ 工具结果总预算	单个结果截到 20000 字符; 多个大结果叠加仍可能偏大 (压缩会兜底但偏被动)	CC 的 applyToolResultBudget: 对「聚合后」的工具结果设总预算，比逐个截断更稳。
+ 只读工具真并发收益验证	StreamingToolExecutor 已并发; 但若 provider 把 tool_calls 一次性吐出，并发收益取决于工具自身耗时	对 read_file/glob/grep/web_fetch 这类无副作用、可能慢的只读工具，并发收益最大——已天然受益于现机制。

取舍没变，只是天平更靠中间了。 ai 依然不打算抄 Claude Code 那 1730 行的全部容错；但 v2 证明：流式、流式并发、总结式压缩这三招，各自只要几十行就能落进一个仍然「一眼读懂」的循环里。剩下的 ④⑥ 属于「多 provider / 多并发会话」才需要的税，按需再加。

依据源码：`ai/src/agent/engine.ts` (runAgent + StreamingToolExecutor)、`ai/src/agent/compact.ts`、`ai/src/llm.ts` (streamCompletion)、`ai/src/tools.ts`、`ai/src/agent/session.ts`；`claudecode/query.ts` (queryLoop)；`openclaw-2026.3.23/src/agents/pi-embedded-runner/run.ts` 及 `compact.ts`。OpenClaw 的真正 agent 循环来自 Pi 引擎（开源于 github.com/earendil-works/pi），发布为 `@mariozechner/pi-agent-core / pi-coding-agent / pi-ai`）；本轮已据该仓库源码 `packages/agent/src/agent-loop.ts` 与 `packages/coding-agent/src/core/compaction/compaction.ts` 核实其双层循环、工具并发时机与压缩阈值，不再仅凭调用壳推断。本页为 v2，与 v1 ([对比-agent-loop-三方.html](#)) 并存、未覆盖原文件。